LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior backend engineer with 9+ years building **distributed systems** and **cloud-native** platforms; in **FinTech / regulated payments**, I delivered Kotlin (**v1.4–v1.9**) and JVM (**11–17**) services on AWS with **Kafka** (**v2.6–v3.6**) and **PostgreSQL** (**v11–v16**) to unify fragmented regional account systems into a resilient global, event-driven backbone. I specialize in **event modelling**, **data consistency**, and **reliability engineering**, using **gRPC** (**v1.30–v1.60**) / REST, **Kubernetes** (**v1.18–v1.29**), and **Terraform** (**v0.12–v1.6**) to ship secure, observable infrastructure that other teams build on top of.

SKILLS

- **Backend: Kotlin v1.4–v1.9**, JVM 11–17, Spring Boot **v2.3–v3.2**, Coroutines/Flow, Gradle, REST, **gRPC v1.30–v1.60**
- **Messaging & Streaming: Kafka v2.6–v3.6**, Schema evolution, idempotency, retries/DLQ patterns, event modelling, async workflows
- **Data: PostgreSQL v11–v16**, relational modelling, transactions, indexing, partitioning, CDC concepts, read/write patterns for consistency
- **Cloud & Platform:** AWS (EKS/ECS/EC2/IAM/RDS/S3/CloudWatch), **Kubernetes v1.18–v1.29**, Docker, **Terraform v0.12–v1.6**, Helm, GitOps basics
- **Reliability & Quality:** SLOs, incident response, load testing, chaos-style failure drills, tracing/metrics/logging, contract tests, CI/CD pipelines
- **Security / Compliance:** least-privilege IAM, secrets management, audit logging, encryption-in-transit/at-rest, change control in regulated environments

WORK HISTORY

Senior Software Engineer
August 2021 to November 2025

RavenTrack

- I designed **core account services** in **Kotlin** using Spring Boot (**v2.6–v3.2**) and Coroutines to handle high-volume money movement with strict correctness rules.
- I led a multi-region consolidation where fragmented account ledgers were replaced by a unified **event-driven** domain model built on **Kafka** (**v2.8–v3.6**).
- I introduced **gRPC** (**v1.40–v1.60**) for internal service-to-service calls to cut tail latency and remove brittle REST fan-outs in critical flows.
- I solved recurring "double-posting" incidents by enforcing **idempotency keys**, transactional outbox, and consumer de-duplication with strong audit trails.
- I migrated legacy regional batch settlement into streaming processors, carefully handling **ordering**, **retries**, and **DLQ** recovery to prevent silent data drift.
- I tackled consistency bugs caused by cross-region reads by implementing write fences, monotonic versioning, and compensating workflows for edge cases.
- I improved database performance on **PostgreSQL** (**v12–v16**) by redesigning hot tables, adding partitioning, and tuning indexes for ledger lookups.
- I built a reusable internal platform layer that standardized **event schemas**, validation, and backward-compatible evolution to protect downstream teams.
- I moved services from ad-hoc VMs to **Kubernetes** (**v1.21–v1.29**) on AWS EKS, adding readiness probes, HPA policies, and safe rollout strategies.
- I implemented infra with **Terraform** (**v0.14–v1.6**) to codify IAM, RDS, EKS, and networking, reducing environment drift and review time.
- I resolved production "stuck consumer" failures by fixing offset commit timing, introducing backpressure, and adding consumer lag alerting with runbooks.
- I built deep observability with distributed tracing and structured logs, making multi-hop payment investigations possible within minutes, not hours.
- I handled version upgrades safely, including Kotlin (**v1.5→v1.9**) and Spring Boot (**v2→v3**) migrations, removing deprecated APIs and tightening tests.
- I partnered with security/compliance to implement **audit logging**, least-privilege access, and change controls suitable for regulated banking operations.
- I mentored engineers on event modelling and failure modes, setting a craftsmanship bar with design docs, reviews, and pragmatic iteration.

Software Engineer
June 2018 to July 2021

Infinite Loop

- I built backend APIs in **Kotlin** and Java on the JVM (**11–17**), focusing on reliability patterns like timeouts, retries, and bulkheads.
- I implemented asynchronous processing with **Kafka** (**v2.6–v3.1**) to decouple services and eliminate cascading failures during peak traffic.
- I diagnosed a hard-to-reproduce race condition in account state transitions by adding deterministic tests and tightening transactional boundaries.

- I introduced **contract testing** for REST/gRPC integrations, preventing breaking changes from silently shipping across dependent teams.
- I migrated an older synchronous integration into an event-stream workflow, adding replay tooling to safely backfill historical events.
- I improved **PostgreSQL** query stability by adding targeted indexes, rewriting joins, and tracking query plans for regressions after releases.
- I containerized services with Docker and deployed to **Kubernetes** (**v1.18–v1.24**), standardizing configs and rollout health checks.
- I wrote Terraform (**v0.12–v1.3**) modules for repeatable environments, reducing manual console changes and making reviews auditable.
- I fixed production 502/timeout spikes by tuning connection pools, adding caching where safe, and reducing N+1 downstream calls.
- I established on-call runbooks and alert thresholds for Kafka consumer lag and DB saturation, improving incident response confidence.
- I delivered internal libraries for logging, correlation IDs, and error mapping so teams shipped consistent, debuggable services.

Software Developer
October 2015 to May 2018

Datamart

- I developed backend systems on the JVM and relational databases, building strong fundamentals in transactional integrity and data modelling.
- I maintained legacy services and reduced recurring defects by refactoring error-prone modules into testable components with clearer boundaries.
- I resolved a long-running "partial update" bug by enforcing atomic operations and adding validation to block inconsistent writes early.
- I optimized reporting workloads on relational databases by batching queries, introducing proper indexing, and reducing lock contention.
- I helped migrate older deployments into container-based workflows, documenting steps and automating builds to reduce human error.
- I implemented resilient background jobs with retry policies and dead-letter handling to prevent silent failure and data loss.
- I improved API stability by standardizing error responses, input validation, and backward-compatible changes for client integrations.
- I added structured logging and basic metrics so failures could be traced across modules instead of relying on guesswork.
- I collaborated closely with QA and product, translating ambiguous requirements into deterministic acceptance criteria and reproducible tests.

EDUCATION

Bachelor's degree in Computer Science
2011 to 2015

University of Amsterdam

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

Global Account Unification Backbone

- Designed a multi-region **Kafka** event model for accounts and balances, including schema evolution rules and replay tooling to safely migrate legacy regions.
- Built a Kotlin **gRPC** platform service that enforces idempotency, audit logging, and consistency checks, enabling teams to build new banking capabilities on top.